

PROGRAMOVATEĽNÉ LOGICKÉ AUTOMATY

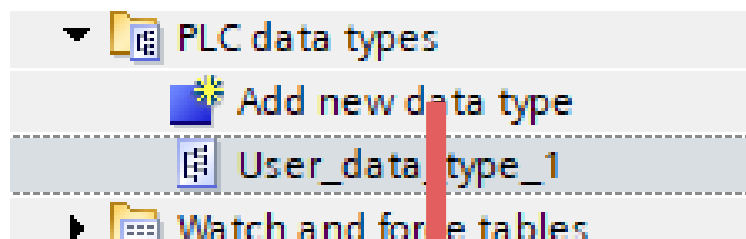
Prednáška #9

OBSAH

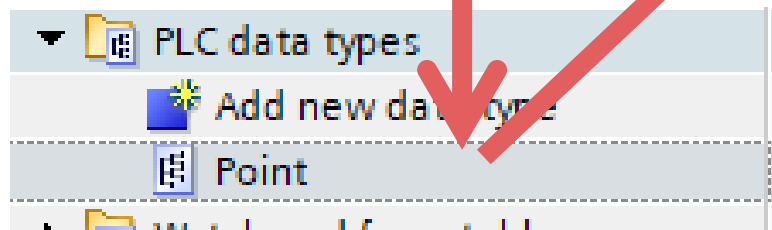
- Užívateľské dátové typy (UDT) – (v TIA: PLC Data Types)
- Dátové bloky (DB)
- Funkčné bloky (FB)
 - Multiinštalácie

Užívateľské dátové typy – UDT (1/7)

- Data type



F2 - premenovať



Point					
		Name	Data type	Offset	Default value
1		X	Real	...	0.0
2		Y	Real	...	0.0
3		<Add new>			

Uživateľské dátové typy – UDT (2/7)

- Úsečka: (Line segment)

LineSegment				
		Name	Data type	Offset
1		▶ A	"Point" 	0.0
2		▼ B	"Point"	8.0
3		■ X	Real	8.0
4		■ Y	Real	12.0

Užívateľské dátové typy – UDT (3/7)

→ FC1 – výpočet dĺžky úsečky

→ DB1 – zoznam úsečiek

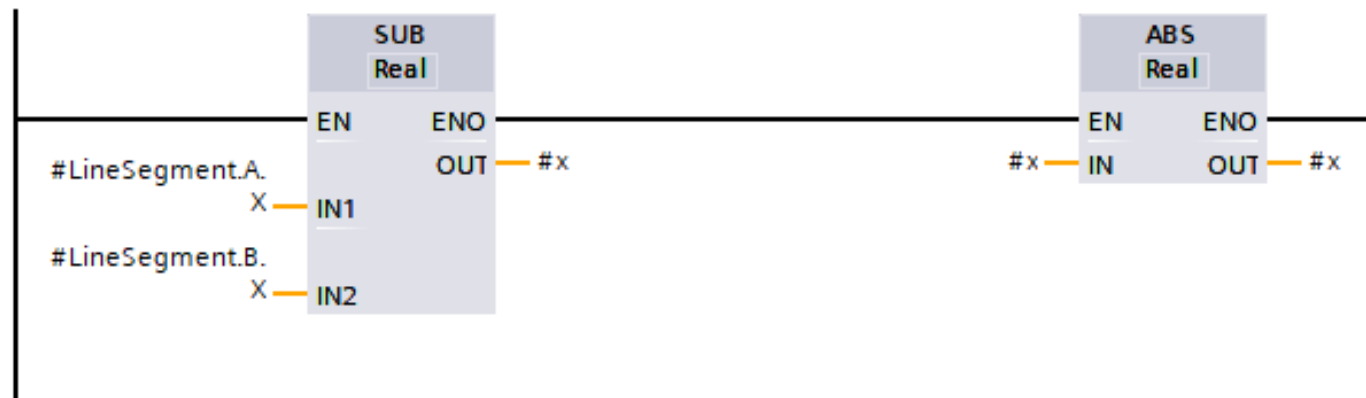
Length			
		Name	Data type
1	☐	▼ Input	
2	☐	▸ LineSegment	"LineSegment"
3	☐	▸ Output	
4	☐	▸ InOut	
5	☐	▼ Temp	
6	☐	▸ x	Real
7	☐	▸ y	Real
8	☐	▸ Constant	
9	☐	▼ Return	
10	☐	▸ Length	Real

Interface funkcie „Length“

LineSegments			
		Name	Data type
	☐	▼ Static	
	☐	▸ a	"LineSegment"
	☐	▸ b	"LineSegment"
	☐	▼ c	"LineSegment"
	☐	▸ A	"Point"
	☐	▼ B	"Point"
	☐	▸ X	Real
	☐	▸ Y	Real

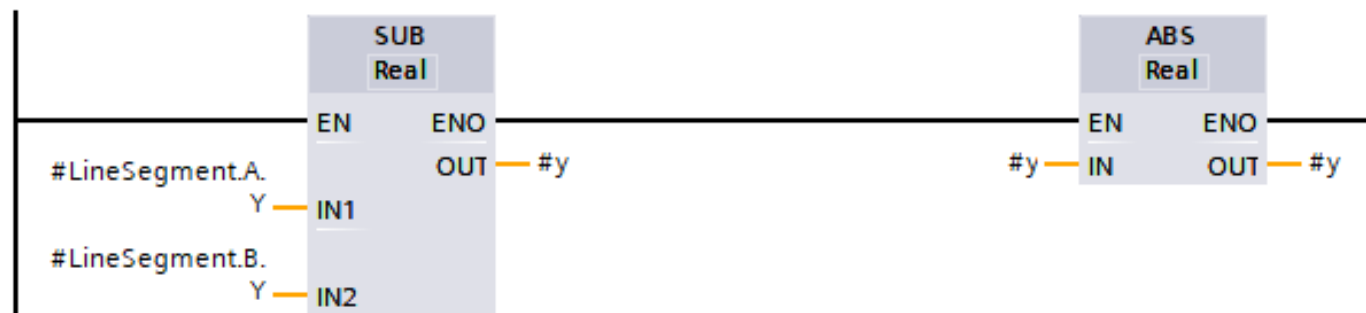
Štruktúra dátového bloku „LineSegments“

Užívateľské dátové typy – UDT (4/7)



▼ Network 2: Vzdielnosť medzi Y súradnicami bodov A a B

Comment



Uživatel'ské datové typy – UDT (5/7)

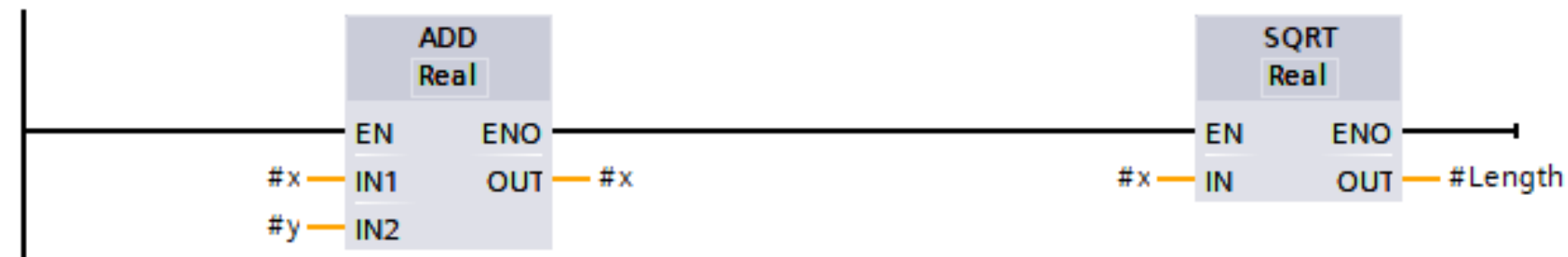
Network 3: Kvadrát X a Y

Comment



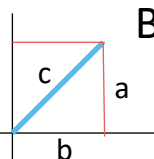
Network 4: $c = \sqrt{x^2 + y^2}$

Comment



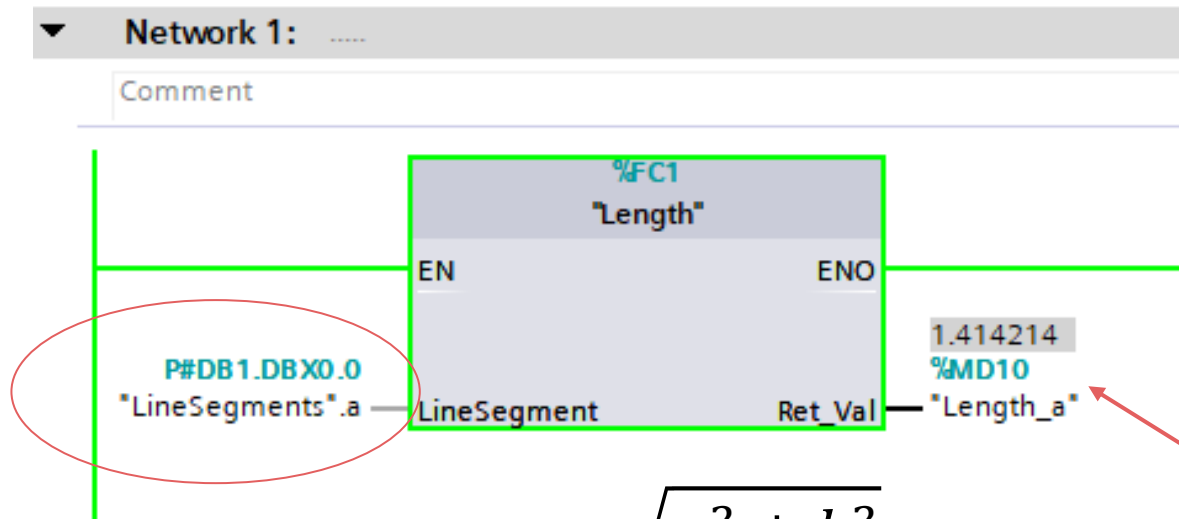
Užívateľské dátové typy – UDT (6/7)

- Príprava dát v dátovom bloku (nastavenie bodov úsečiek)

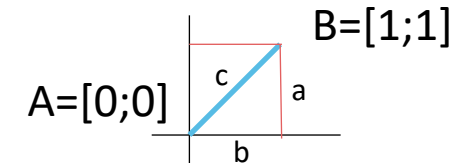
LineSegments					Úsečka a = [A=[0;0]; B=[1;1]]	
		Name	Data type	Offset	Start value	
1	Static					
2	Static	a	"LineSegment"	...		
3	Static	A	"Point"	...		
4	Static	X	Real	...	0.0	
5	Static	Y	Real	...	0.0	
6	Static	B	"Point"	...		
7	Static	X	Real	...	1.0	
8	Static	Y	Real	...	1.0	
9	Static	b	"LineSegment"	...		
10	Static	c	"LineSegment"	...		

Užívateľské dátové typy – UDT (7/7)

- Volanie funkcie „Length“ v OB1



Úsečka a = [A=[0;0]; B=[1;1]]



$$c = \sqrt{a^2 + b^2}$$

$$c = \sqrt{2} = 1,41421$$

$$a = B[y] - A[y] = 1$$

$$b = B[x] - A[x] = 1$$

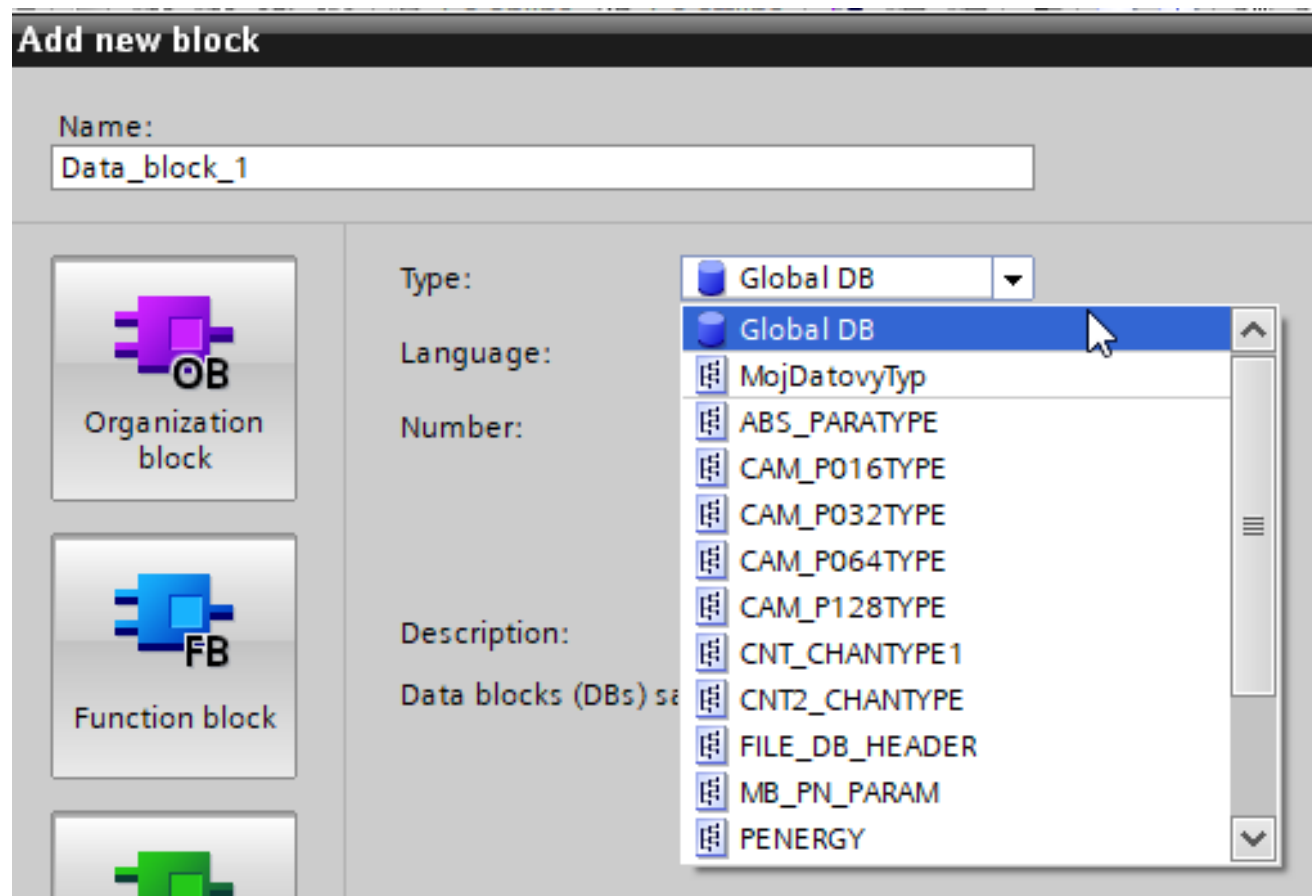
Dátové bloky – DB (1/4)

- **Inštančné DB** – generované automaticky počas volania FB
- **Globálne DB** – vytvorené používateľom
- **Typové DB** – ak máme definované UDT, preberá jeho štruktúru

Dátové bloky – DB (2/4)

- Elementárne DT
- Komplexné DT
- Názov premenných bez úvodzoviek

Data_block_1		
	Name	Data type
1	Static	
2	s medzerou	Bool
3	8 zacina cislom	Bool
4	# zacina znakom	Bool
5	"medzi uvodzovky"	Bool
6	'medzi apostrofy'	Bool
7	premenna\$	Bool
8	* / -	Bool
9	<Add new>	



Dátové bloky – DB (3/4)

Pred kompiláciou

DB1					
		Name	Data type	Offset	Start value
1		▼ Static			
2		■ bool	Bool	...	false
3		■ byte	Byte	...	16#0
4		■ word	Word	...	16#0
5		■ double	DWord	...	16#0
6		▼ Pole	Array[0..127] of Byte	...	
7		■ Pole[0]	Byte	...	16#0
8		■ Pole[1]	Byte	...	16#0
9		■ Pole[2]	Byte	...	16#0
10		■ Pole[3]	Byte	...	16#0

Po kompilácii

DB1					
		Name	Data type	Offset	Start value
1		▼ Static			
2		■ bool	Bool	0.0	false
3		■ byte	Byte	1.0	16#0
4		■ word	Word	2.0	16#0
5		■ double	DWord	4.0	16#0
6		▼ Pole	Array[0..127] of Byte	8.0	
7		■ Pole[0]	Byte	8.0	16#0
8		■ Pole[1]	Byte	9.0	16#0
9		■ Pole[2]	Byte	10.0	16#0
10		■ Pole[3]	Byte	11.0	16#0

Dátové bloky – DB (4/4)

▪ Príklad adresovania:

- PLC S7-300 a S7-400 nepoužívajú optimalizáciu, preto môžeme pristupovať absolútne aj symbolicky.
- PLC S7-1200 a S7-1500 používajú optimalizáciu, prístup je možný len symbolicky (optimalizáciu je možné vypnúť, potom je možný prístup aj absolútne)

Absolútne

1 bit: DB1.DBX2.5
8 bit: DB3.DBB2
16 bit: DB5.DBW6
32 bit: DB3.DBD12

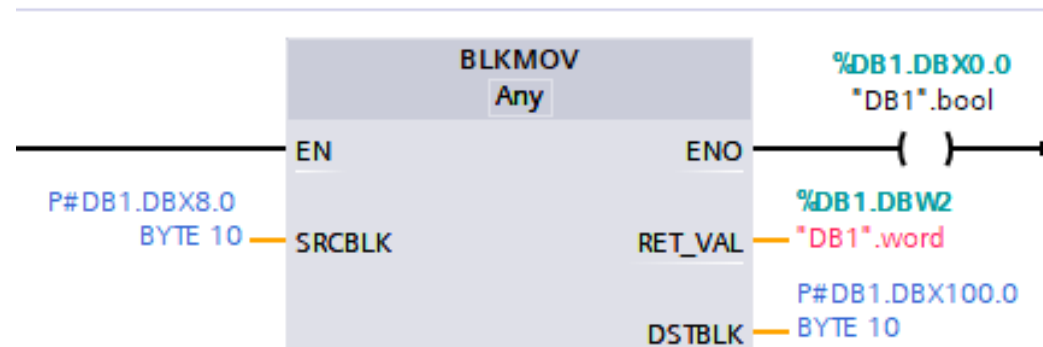
Symbolicky

1bit: "názov_DB".názov_premennej
=> „LineSegments“.a.A.x
UDT: "názov_DB".názov_premennej
=> „LineSegments“.a

Väčšie DT iba cez pointer alebo symbolicky

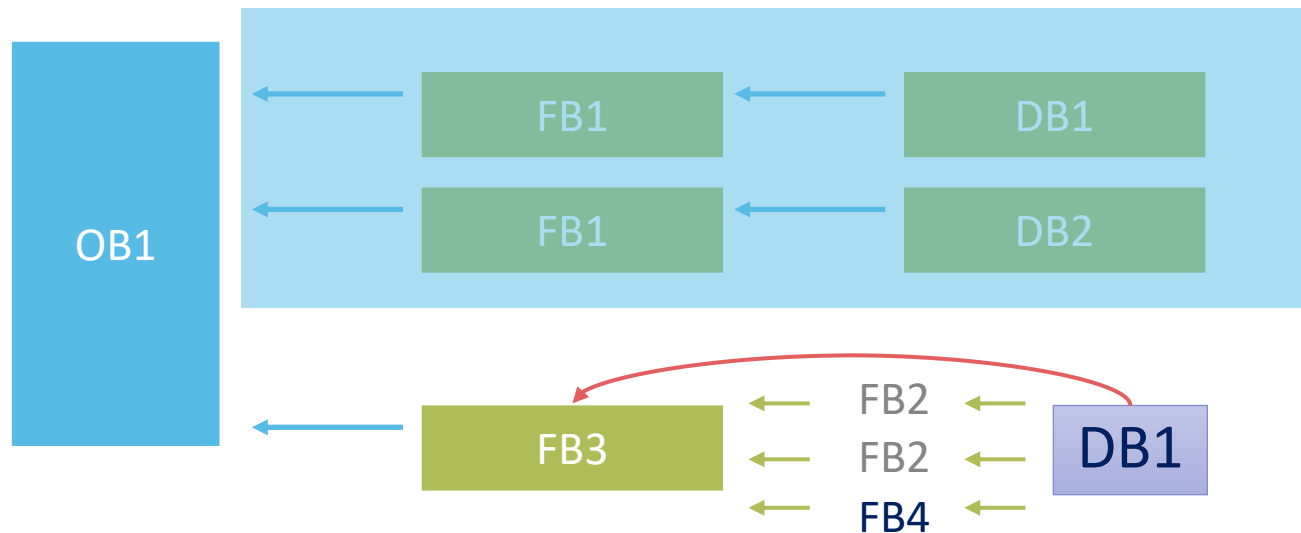
P#DB1.DBX8.0 Byte 10

(10 „bajtov“ od adresy DB1.DBX2.0)



Funkčné bloky – multiinštancie

- Viacnásobné volanie funkčného bloku
- Využitie iba jedného DB

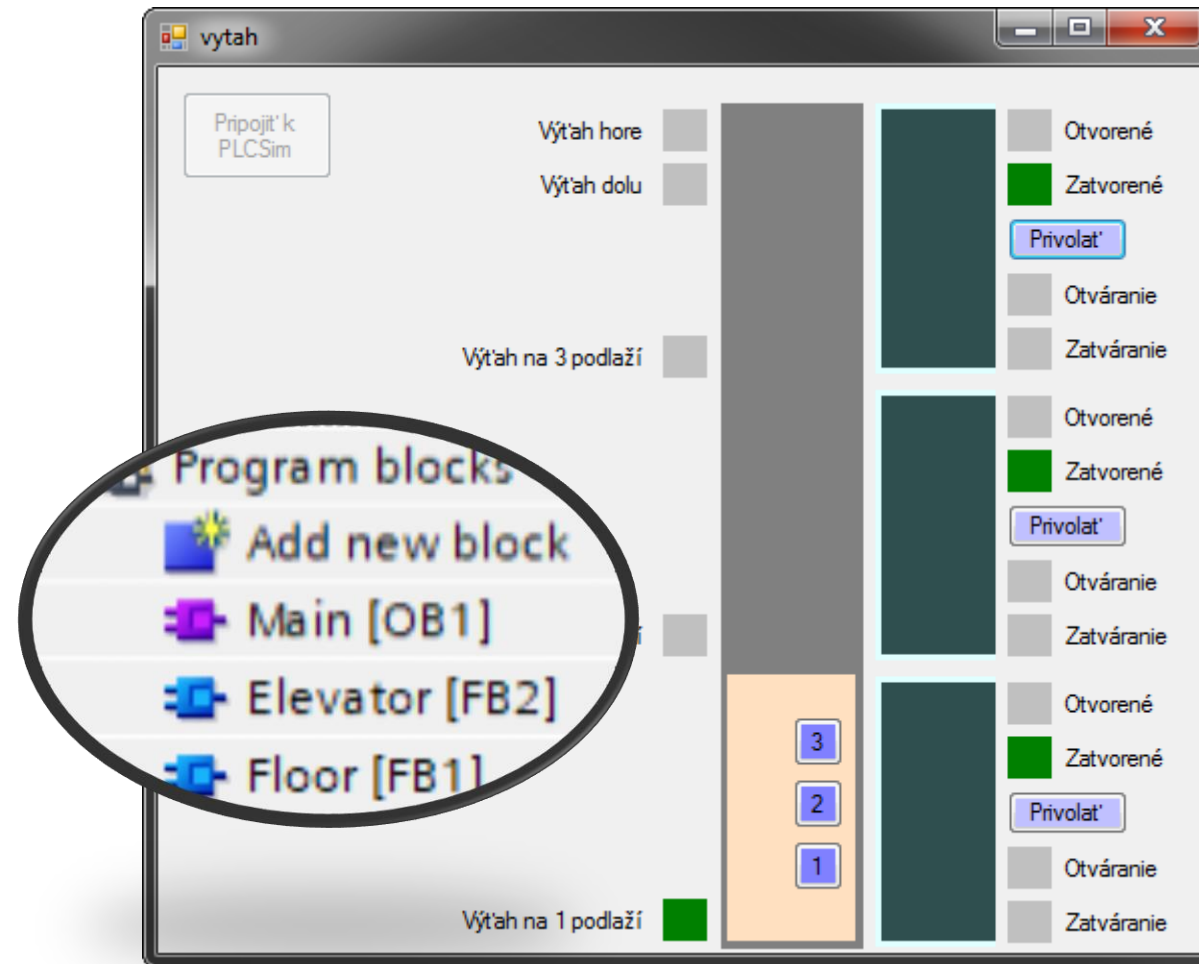
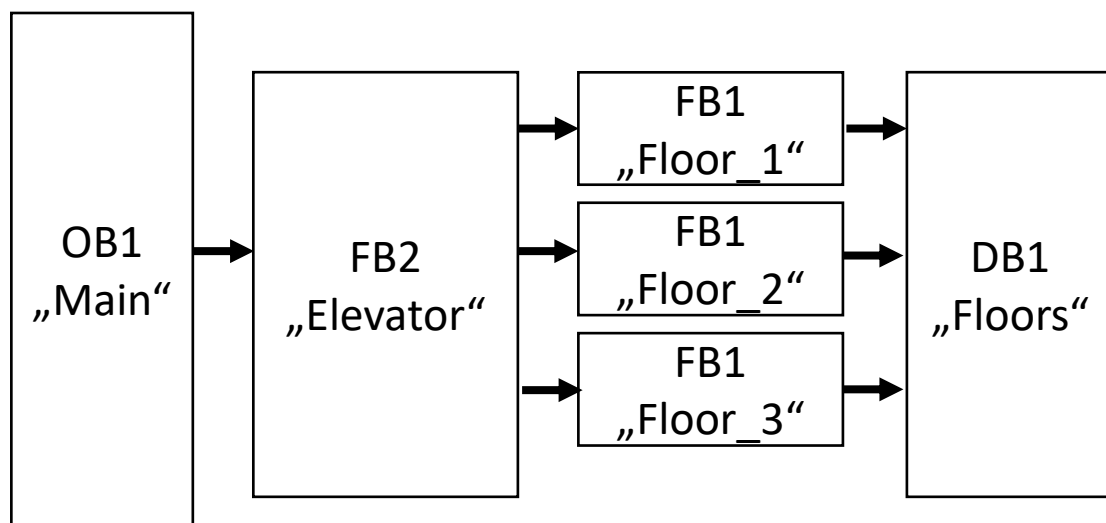


„single“ inštancie

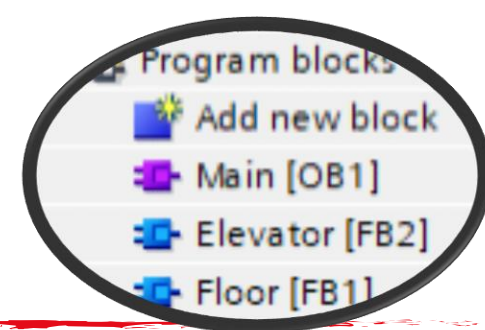
multiinštancie

Príklad – multiinštancie (1/6)

- Výťah
 - Podlažia (FB)
- Štruktúra programu

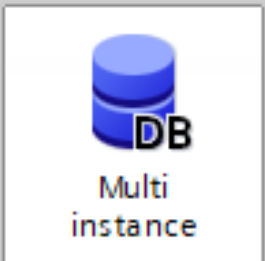
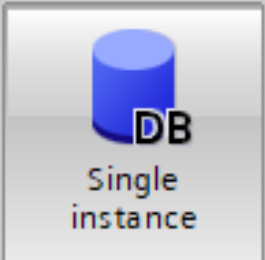


Príklad – multiinštancie (2/6)



- Pri volaní funkčného bloku v inom funkčnom bloku (voláme blok FB1 v bloku FB2) dostaneme od TIA možnosť výberu, zvolíme „Multi instance“

Call options



Multiple instance

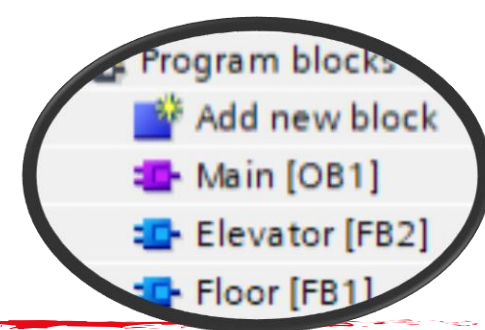
Name in the interface

If you call the function block as a multi-instance, it saves data in the instance data block of the calling function block, not in its own instance data block. This allows you to concentrate the instance data in one block and to get by with fewer instance data blocks in your program.

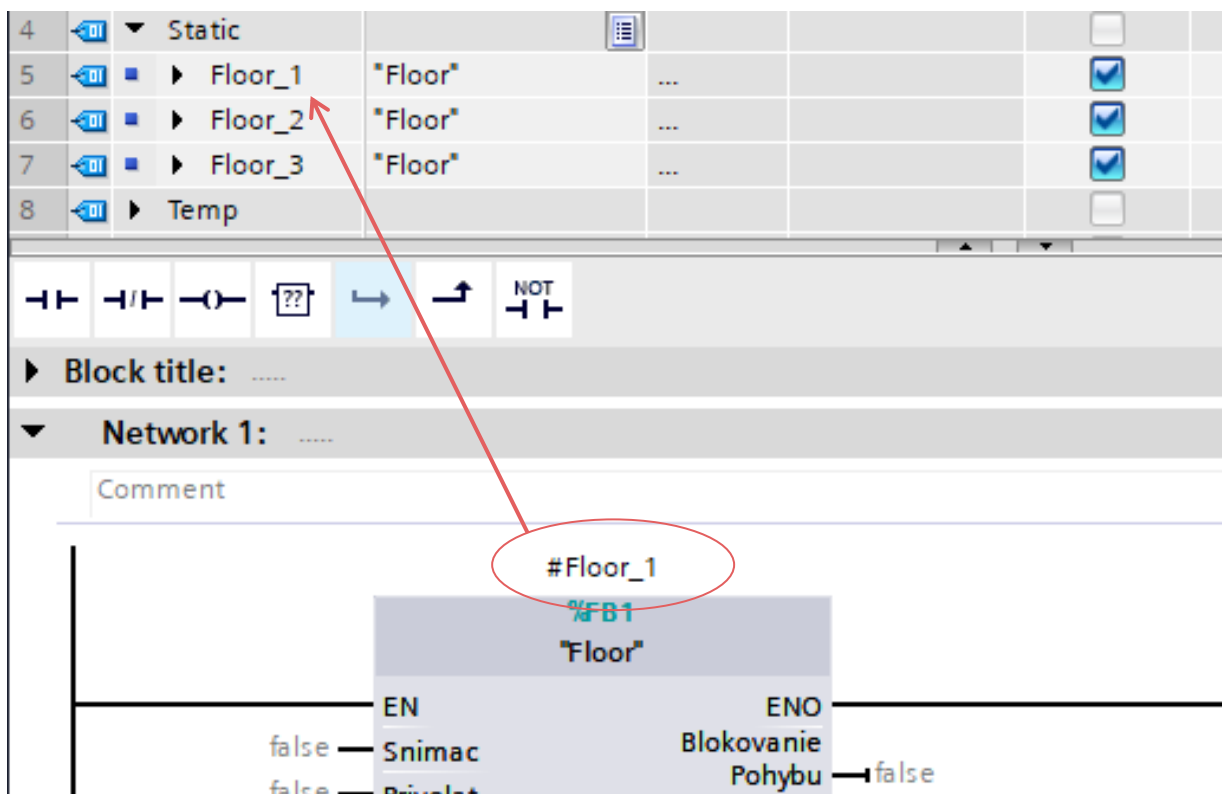
#Floor_1

	%FB1	"Floor"	
	EN	ENO	
false	Snimac	Blokovanie	
false	Privolat	Pohybu	→ false
false	OtvoreneDvere	DvereOtvorit	→ false
	Zatvorene	DvereZatvorit	→ false
false	Dvere	Poziadavka	→ false

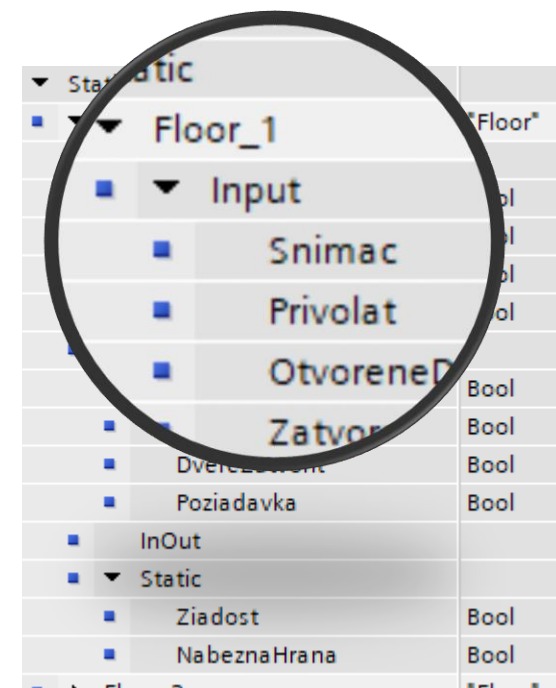
Príklad – multiinštancie (3/6)



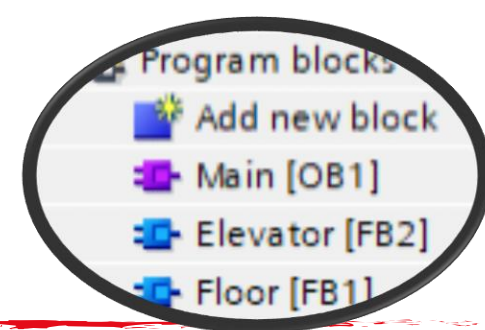
- Volanie FB1 v FB2



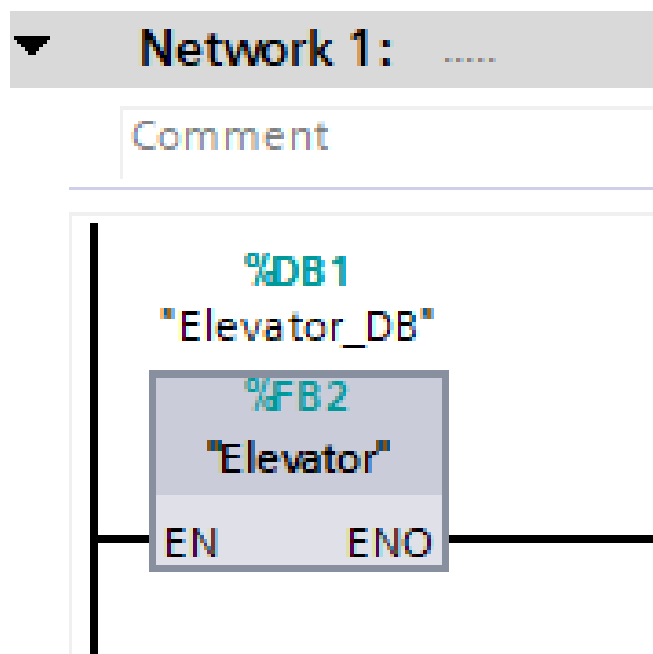
V STATIC premenných pribudnú Floor_1 až Floor_3, ktoré sú dátového typu FB1 „Floor“



Príklad – multiinštancie (4/6)



- Volanie FB2 v OB1



Call options

 DB
Single instance

Data block

Name: Elevator_DB

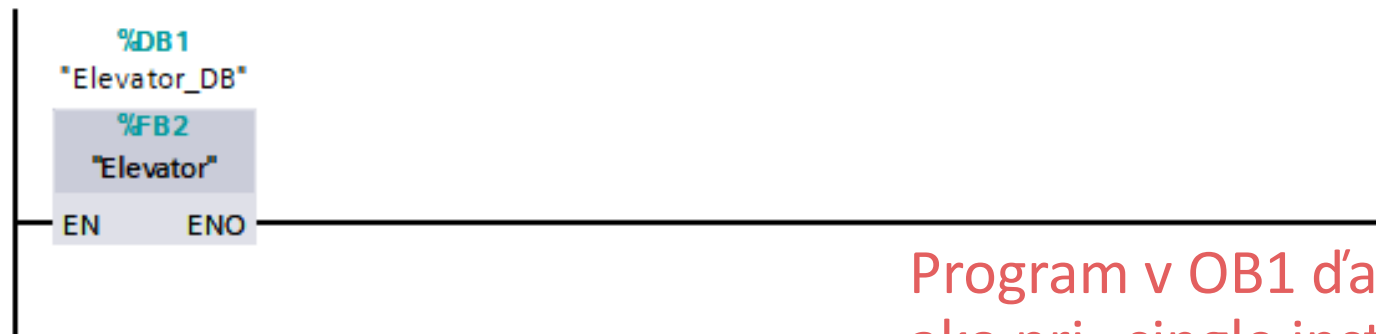
Number: 1

☐ Manual

☒ Automatic

If you call the function block as a single instance, the block saves its data in its own instance data block.

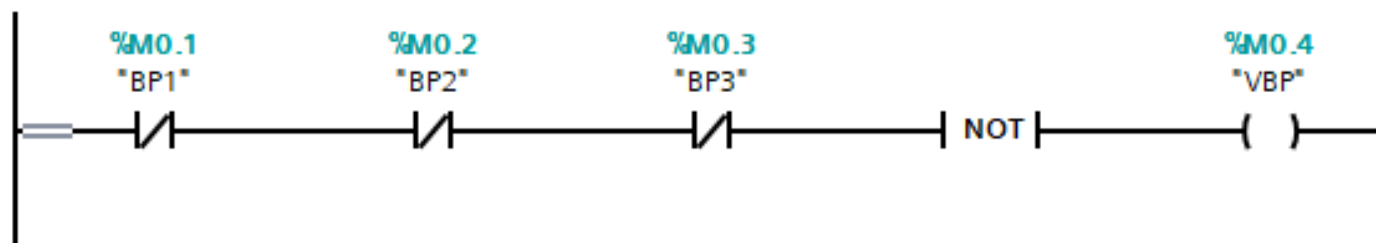
Príklad – multiinštancie (5/6)



Program v OB1 ďalej zostáva rovnaký ako pri „single instance“

Network 2: Vseobecne blokovanie pohybu

Comment

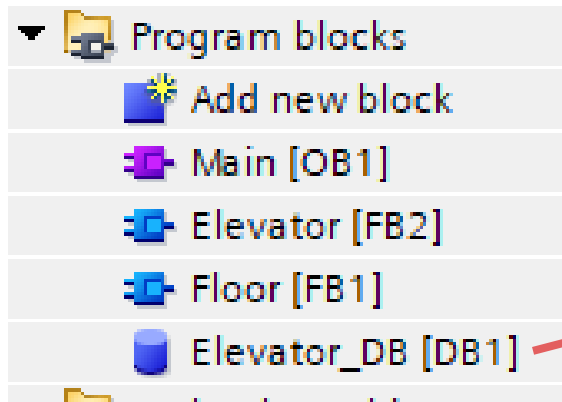


Network 3: Vseobecna poziadavka pohybu

Network 4: Poziadavka - smer hore

Network 5: Poziadavka - smer dole

Príklad – multiinštancie (6/6)



Elevator_DB				
		Name	Data type	Offset
1		Input		
2		Output		
3		InOut		
4		▼ Static		
5		■ ▼ Floor_1	"Floor"	0.0
6		■ ▼ Input		
7		■ Snimac	Bool	0.0
8		■ Privolat	Bool	0.1
9		■ OtvoreneDvere	Bool	0.2
10		■ ZatvoreneDvere	Bool	0.3
11		■ ► Output		
12		■ InOut		
13		■ ► Static		
14		■ ► Floor_2	"Floor"	6.0
15		■ ► Floor_3	"Floor"	12.0

PROGRAMOVATEĽNÉ LOGICKÉ AUTOMATY

Prednáška #9